# More Simulation

(download slides and .py files to follow along!)

John Guttag

MIT Department Of Electrical Engineering and Computer Science

# Rescheduling Things

- Had a snow day on Monday
- Today, will give lecture scheduled for Monday
- Next Monday, instead of a recitation, we will have lecture scheduled for today
- Nothing else changes

MIT News Office
Feb. 2013

# Quiz 1, Question 1, Part 4

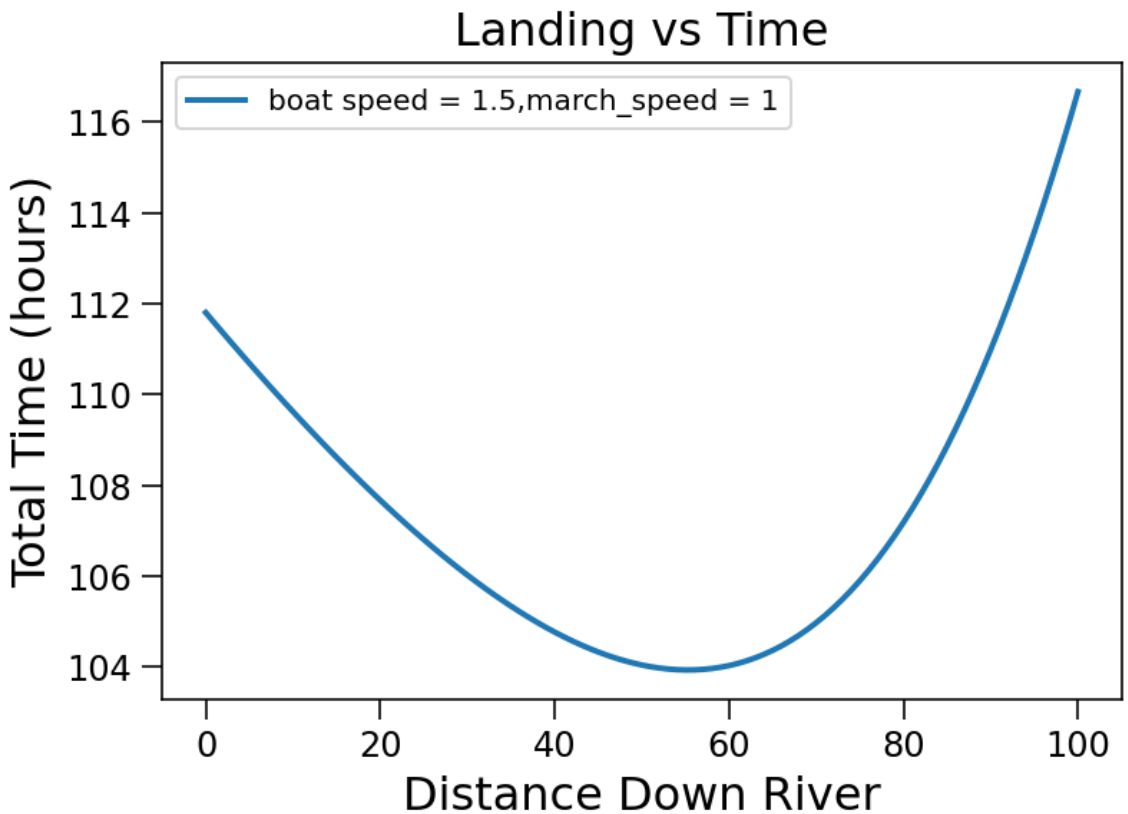
**Question 1 [2 points]:** Consider a modified version of the problem from Lecture 1, where a destination is d_south km and d_east km from the start. There are two components to the travel between start and destination.

The first is by boat on a river that runs directly south of the start. The speed of the boat on the river is first r1 km/h. At some point on the river, r km from the start, the speed changes to r2 km/h, with r2 > r1.

The second component is a constant walking speed w km/h from wherever the boat lands on the river to the destination. For each situation below, choose which are true.

If there are an infinite number of places for the boat to dock, can this problem can be solved using ternary search?

# When Does Ternary Search Work?

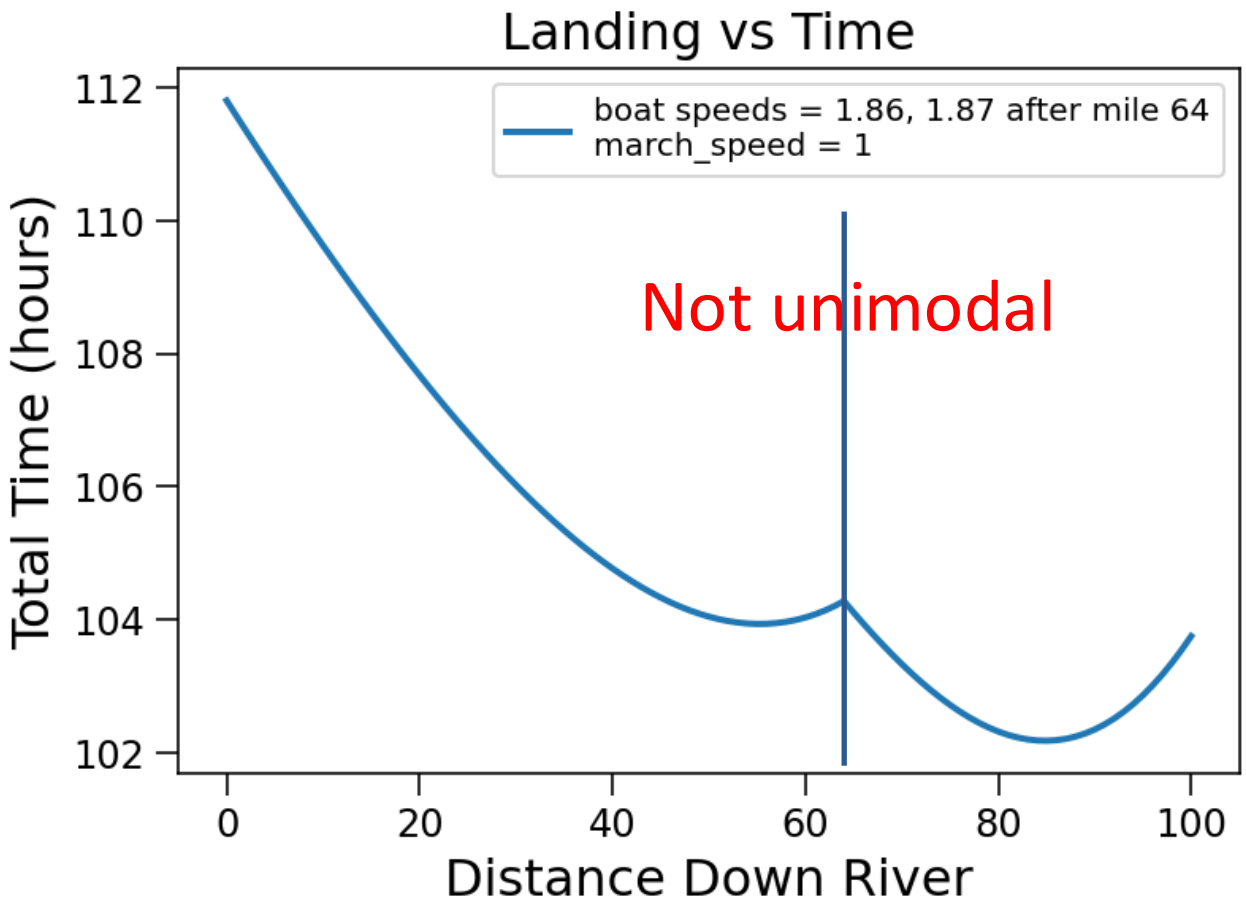



Unimodal

Strictly decreasing, then

Strictly increasing

# Quiz Question About Ternary Search




Gave credit for both answers

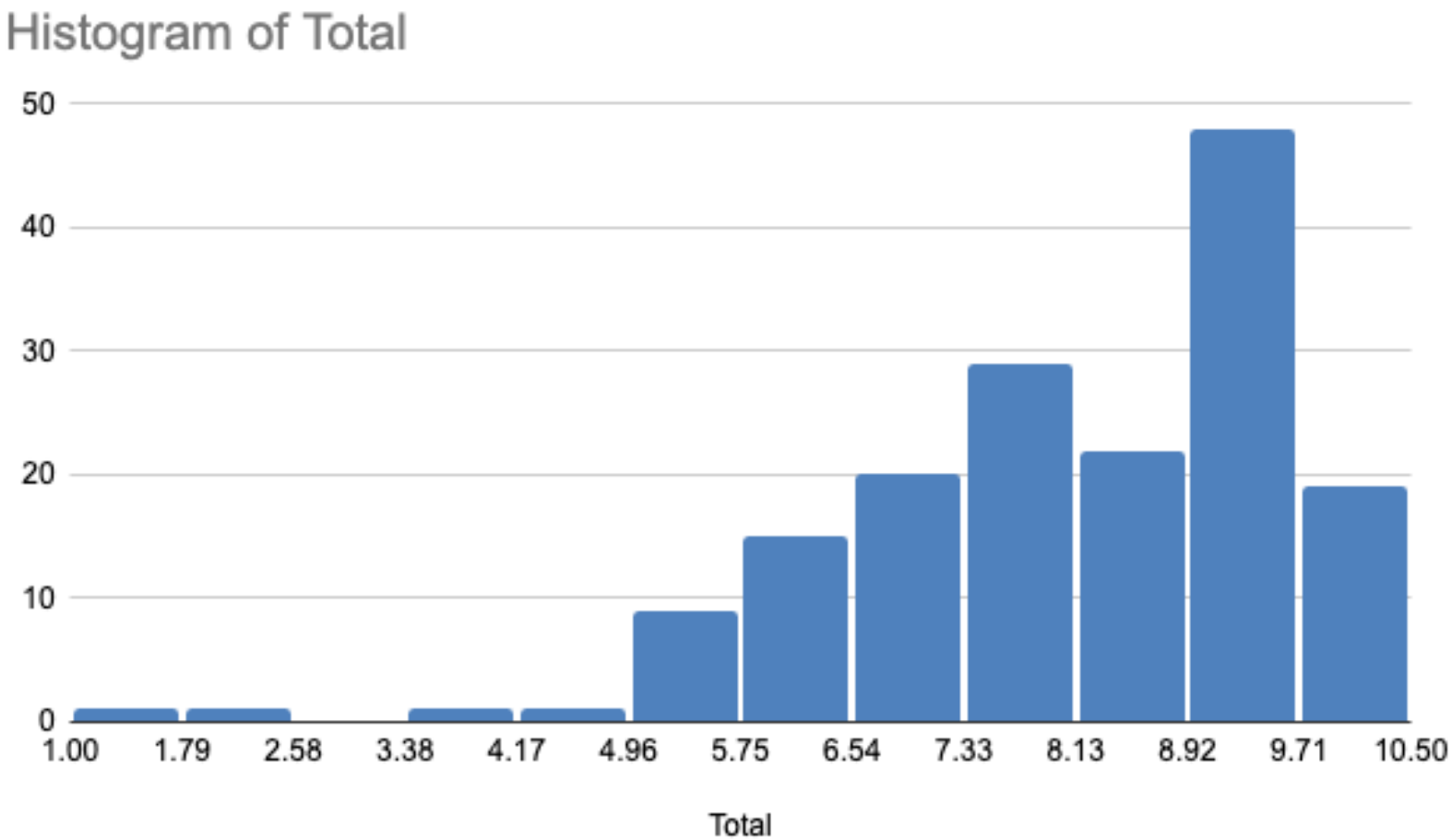
Cannot solve with one ternary search

Can solve piecewise

Use two ternary searches

Take min of two solutions

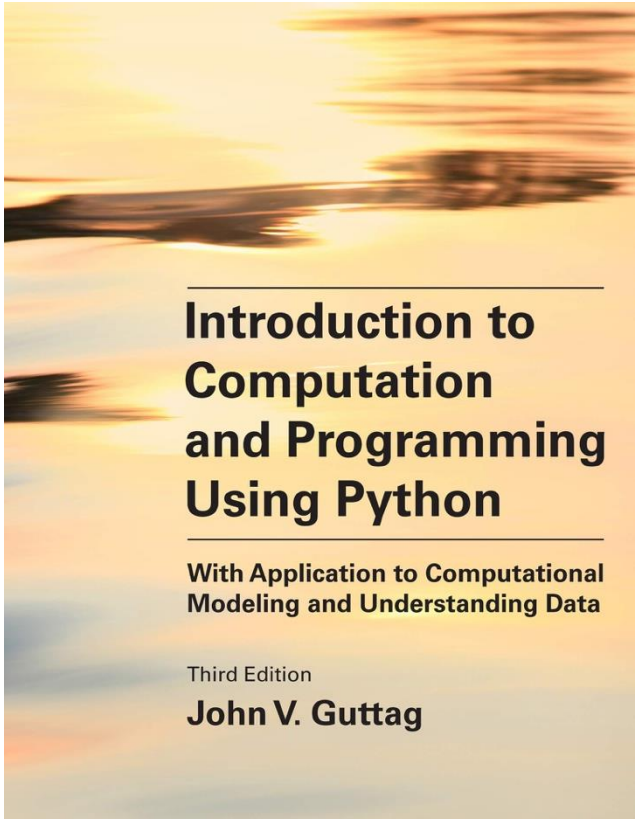
# Quiz 1 Results




Mean: 8.1
Median: 8.5
Standard Deviation: 1.48

# Assigned Reading

- Today:
  - Chapter 16
- Next lecture:
  - Chapter 18




All code and an errata sheet at
https://github.com/guttag/Intro-to-Computation-and-Programming

# Recapping Last Lecture

- Simulations highly useful for understanding stochastic situations
- Roulette is analyzable using probabilities, but most things other than human-designed games of chance are not
- Averages are only part of the story
- Important to understand how possible results are distributed

# Sampling Space of Possible Outcomes

- Never possible to guarantee perfect accuracy through sampling
- Not to say that an estimate is not precisely correct
- Key question:
  - How many samples do we need to look at before we can have justified confidence in our answer?
- Depends upon variability in underlying distribution

# Quantifying Variation in Data

$$variance(X) = \frac{\sum_{x \epsilon X}(x-\mu)^2}{|X|}$$

μ is the mean of $X$

$$\sigma(X) = \sqrt{\frac{1}{|X|} \square_{x \square X} (x-\mu)^2}$$

- Standard deviation simply the square root of the variance
  - `numpy.std`
- Outliers can have a big effect

# Confidence Levels and Intervals

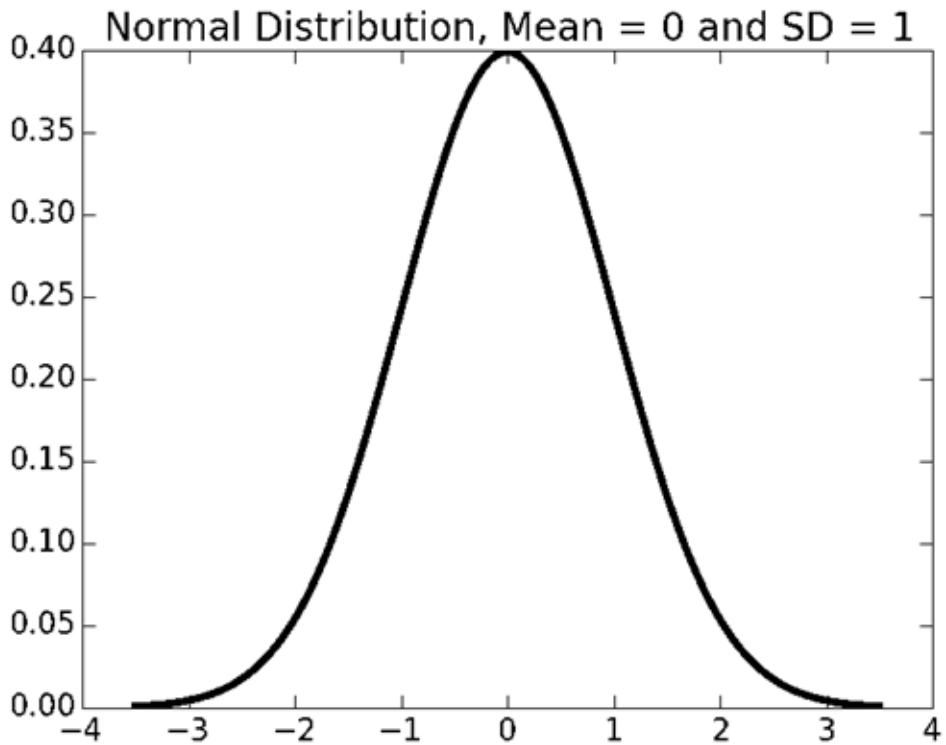
- Instead of estimating an unknown parameter by a single value, a confidence interval provides a range that is likely to contain the unknown value and a confidence that the unknown value lays within that range
- "The estimated return on betting a pocket 10k times in European roulette is -2.7%. The margin of error is +/-1% with a 95% level of confidence."
- What does this mean?
- If I were to repeat the same experiment of 10k bets an infinite number of times:
  - In 95% of those experiments, the generated confidence interval (-3.7% - 1.7%) would include the unknown return

# Empirical Rule

- Where do we get the range of the confidence interval?
- Empirical rule
  - ~68% of data lies within one standard deviation of mean
  - ~95% of data lies within 1.96 standard deviations of mean
  - ~99.7% of data lies within 3 standard deviations of mean

# Assumptions Underlying Empirical Rule

- The mean estimation error is zero
- The distribution of the errors in the estimates is normal (but works pretty well for other distributions)




More about distributions in an upcoming lecture

# Applying Empirical Rule

```
def sim_game(game, spin_list, num_trials):
    means, stds = [], []
    for num_spins in spin_list:
        print(f'Simulate {num_trials:,} trials of {num_spins:,}'
              f' spins each of {game.__name__}')
        pocket_returns = [play_roulette(game(), num_spins, pocket=2, bet=1)
                          for _ in range(num_trials)]
        mean = sum(pocket_returns)/len(pocket_returns)
        means.append(mean)
        std_dev = np.std(pocket_returns)
        stds.append(std_dev)
        print(f'Estimated return = {mean:,}'
              f' ±{round(1.96*std_dev):,} with 95% confidence')
    return means, stds
```

Run multiple trials of estimating roulette's return to get standard deviation of that estimate

# Results

Simulate 20 trials of 10,000 spins each of Fair_roulette
Estimated return = 18.8 ±1,222 with 95% confidence
Simulate 20 trials of 100,000 spins each of Fair_roulette
Estimated return = -200.8 ±3,349 with 95% confidence
Simulate 20 trials of 1,000,000 spins each of Fair_roulette
Estimated return = -787.6 ±10,854 with 95% confidence

Simulate 20 trials of 10,000 spins each of Eu_roulette
Estimated return = -281.8 ±982 with 95% confidence
Simulate 20 trials of 100,000 spins each of Eu_roulette
Estimated return = -2,951.2 ±3,469 with 95% confidence
Simulate 20 trials of 1,000,000 spins each of Eu_roulette
Estimated return = -28,549.0 ±12,024 with 95% confidence

Should we be troubled by ever increasing confidence intervals?

# Interpreting Variance

- Standard deviation (which is used to compute CI) must always be understood relative to mean
- Here means increase because number of bets increase
- To compare variance when means are different, use coefficient of variation (CV) rather than standard deviation
  - CV = std/abs(mean)

# Revised sim_game

```
def sim_game(game, spin_list, num_trials, std_or_cv = 'std'):
    means, stds = [], []
    for num_spins in spin_list:
        print(f'Simulate {num_trials:,} trials of {num_spins:,}'
              f' spins each of {game.__name__}')
        pocket_returns = [play_roulette(game(), num_spins, pocket=2, bet=1)
                          for _ in range(num_trials)]
        mean = sum(pocket_returns)/len(pocket_returns)
        means.append(mean)
        std_dev = np.std(pocket_returns)
        stds.append(std_dev)
        if std_or_cv == 'std':
            print(f'Estimated return = {mean:,}'
                  f' ±{round(1.96*std_dev):,} with 95% confidence')
        else:
            print(f'Estimated return = {mean:,}'
                  f' ±{round(1.96*std_dev):,} with 95% confidence. '
                  f'CV = {std_dev/abs(mean):.2f}')
    return means, stds
```

# Results

Simulate 20 trials of 10,000 spins each of Fair_roulette
Estimated return = 18.8 ±1,222 with 95% confidence. CV = 33.17
Simulate 20 trials of 100,000 spins each of Fair_roulette
Estimated return = -200.8 ±3,349 with 95% confidence. CV = 8.51
Simulate 20 trials of 1,000,000 spins each of Fair_roulette
Estimated return = -787.6 ±10,854 with 95% confidence. CV = 7.03

Simulate 20 trials of 10,000 spins each of Eu_roulette
Estimated return = -281.8 ±982 with 95% confidence. CV = 1.78
Simulate 20 trials of 100,000 spins each of Eu_roulette
Estimated return = -2,951.2 ±3,469 with 95% confidence. CV = 0.60
Simulate 20 trials of 1,000,000 spins each of Eu_roulette
Estimated return = -28,549.0 ±12,024 with 95% confidence. CV = 0.21

CV moves in expected direction

Is estimated return getting more accurate?

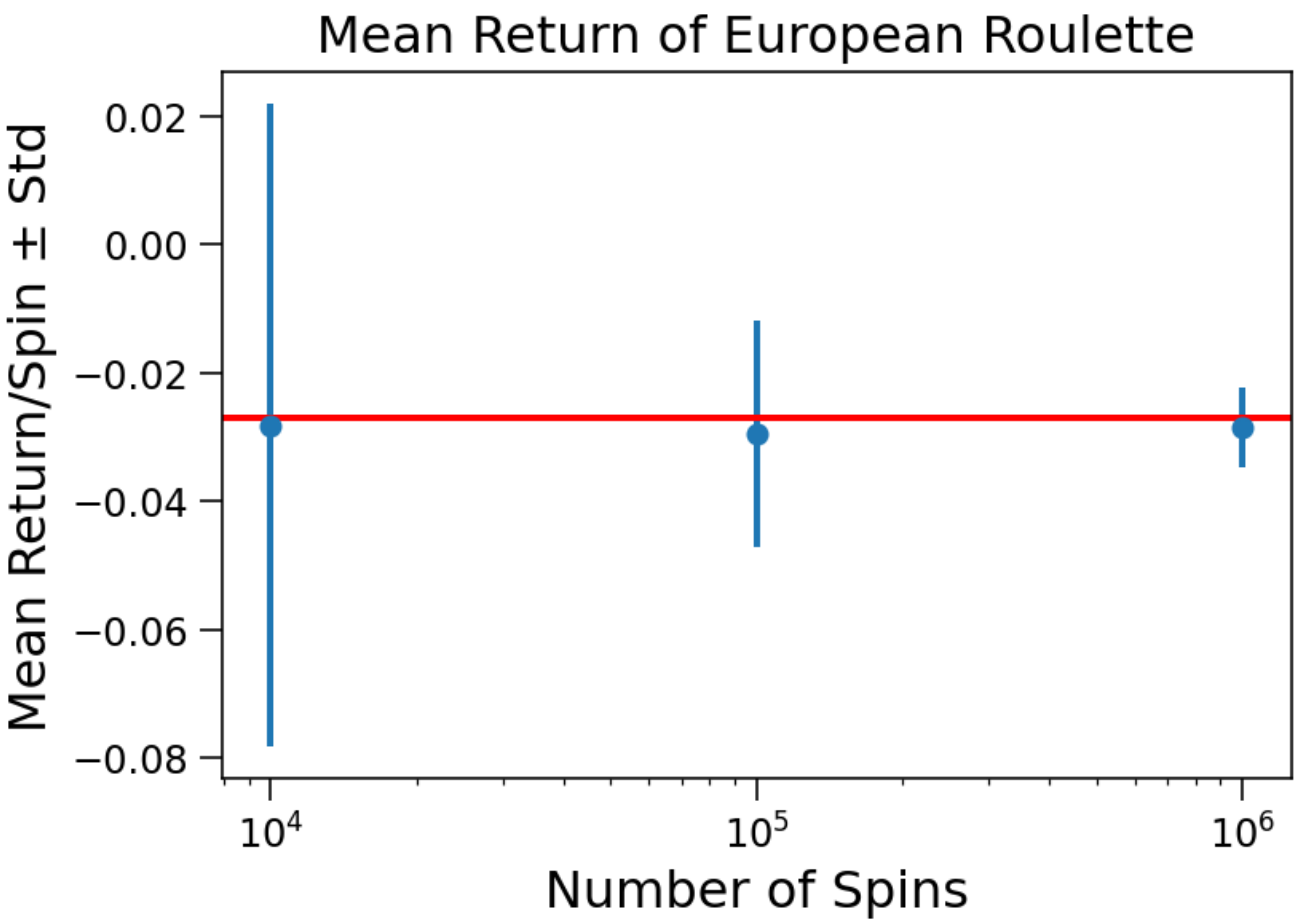
# Estimated Return vs Analytic Return Per Spin

| Fair Roulette | 10k spins | 100k spins | 1M spins | Analytic |
|---|---|---|---|---|
| | 0.0018 | -0.0020 | -0.0008 | 0 |

| European Roulette | 10k spins | 100k spins | 1M spins | Analytic |
|---|---|---|---|---|
| | -0.0282 | -0.0295 | -0.0285 | -0.0270 |

Should we be troubled that increasing number of spins doesn't always lead to better estimate?

# No, It's About Increased Confidence



# A Last Word About Roulette

- It’s a game for suckers
- Don’t play it!

# Simulating Something More Complicated than Roulette

- What is the probability of at least two people in a group having the same birthday?
- If there are 30 people in a room, should you be surprised if two share a birthday?

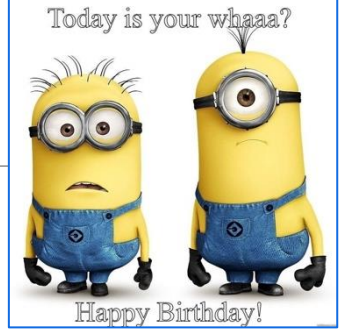
Poll:
What is the probability of two people, out of a group of 30, sharing a birthdate?

~.05
~.1
~.25
~.5
~.7

# The Birthday Problem

- What if there are 367 people in the group?
  - Use the “pigeonhole principle” – if you try to put m items into n containers and m > n, then at least one container must have more than 1 item
- What about fewer then 366 people in the group?

# The Birthday Problem




- If assume each birthdate equally likely, then can use "inclusion/exclusion principle"
- Probability that n (<367) people each have **different** birthday

$$\frac{365}{366} * \frac{364}{366} * \cdots * \frac{366-n+1}{366} = \frac{366!}{366^n * (366-n)!}$$

- Probability that at least two of n people have same birthday:

$$1 - \frac{366!}{366^n * (366-n)!}$$

For n = 23 this is about .5063

For n = 30 this is about .7053

- We play this complement trick a lot with probabilities
  - Often easier to directly derive probability of not(x) than of x

# Because I'm Too Lazy to Compute Probabilities by Hand

```
def true_prob(num_people):
    # Assumes each birthdate is equally probable
    unique_possibilities = math.factorial(366)
                           // math.factorial(366 - num_people)
    total_possibilities = 366**num_people
    return 1 - unique_possibilities / total_possibilities


for i in range(1, 9):
    n = 2**i
    print(f"Actual prob. of a shared birthday "
          f"with {n} people = {true_prob(n):.3f}")
```

# Results

```
Actual prob. of a shared birthday with 2 people = 0.003
Actual prob. of a shared birthday with 4 people = 0.016
Actual prob. of a shared birthday with 8 people = 0.074
Actual prob. of a shared birthday with 16 people = 0.283
Actual prob. of a shared birthday with 32 people = 0.752
Actual prob. of a shared birthday with 64 people = 0.997
Actual prob. of a shared birthday with 128 people = 1.000
Actual prob. of a shared birthday with 256 people = 1.000
```

Really?

# Simulating the Birthday Problem

One trial

```
def same_date(num_people, num_same, birthday_probs):
    possible_dates = range(366)
    birthday_counts = [0] * 366
    birth_dates = np.random.choice(possible_dates, size = num_people,
                                   p = birthday_probs)
    for birth_date in range(len(birth_dates)):
        birthday_counts[birth_dates[birth_date]] += 1
    return max(birthday_counts) >= num_same
```

No births on any day

Choose dates at random

Incr. births

```
def birthday_prob(num_people, num_same, birthday_probs, num_trials):
    num_hits = 0
    for _ in range(num_trials):
        if same_date(num_people, num_same, birthday_probs):
            num_hits += 1
    return num_hits / num_trials
```

Multiple trials

Count number of trials that succeed

Empirical probability

# Simulating the Birthday Problem

```
birth_dates = np.random.choice(possible_dates, size = num_people,
                               p = birthday_probs)
```

numpy.random offers richer set of operations than random
Used by many libraries

Different seed, beware

```
np.random.seed(0)
birthday_probs = np.array([1 for _ in range(366)])/366
num_same = 2
for i in range(1, 9):
    num_people = 2**i
    print(f'Est. prob. of {num_same} people sharing a birthday, '
      f'with {num_people} people: '
      f'{birthday_prob(num_people, num_same, birthday_probs, 10_000):.3f}')
```

Each day same prob.

# Compare to Analytic Solution (10,000 trials)

```
Actual prob. of a shared birthday with 2 people = 0.003
Actual prob. of a shared birthday with 4 people = 0.016
Actual prob. of a shared birthday with 8 people = 0.074
Actual prob. of a shared birthday with 16 people = 0.283
Actual prob. of a shared birthday with 32 people = 0.752
Actual prob. of a shared birthday with 64 people = 0.997
Actual prob. of a shared birthday with 128 people = 1.000
Actual prob. of a shared birthday with 256 people = 1.000
```

```
Est. prob. of 2 people sharing a birthday, with 2 people: 0.003
Est. prob. of 2 people sharing a birthday, with 4 people: 0.019
Est. prob. of 2 people sharing a birthday, with 8 people: 0.075
Est. prob. of 2 people sharing a birthday, with 16 people: 0.278
Est. prob. of 2 people sharing a birthday, with 32 people: 0.748
Est. prob. of 2 people sharing a birthday, with 64 people: 0.997
Est. prob. of 2 people sharing a birthday, with 128 people: 1.000
Est. prob. of 2 people sharing a birthday, with 256 people: 1.000
```

Now that we have a simulation that seems to work for the simple birthday problem, let's look at variants

# How Often Do N People Share a Birthday?

- Suppose we want the probability of 3 people sharing a birthday, out of some group of size N?
- For 2 people, the complementary problem is “all birthdays are distinct”
- For 3 people, the complementary problem is a complicated disjunction of cases, hard, but possible, to describe analytically
- For 4 people, much worse
- How easy is it to use simulation to answer that question?

# Really Easy!

```
birthday_probs = np.array([1 for _ in range(366)])/366
num_same = 4
for i in range(1, 9):
    num_people = 2**i
    print(f'Est. prob. of {num_same} people sharing a birthday, '
      f'with {num_people} people: '
      f'{birthday_prob(num_people, num_same, birthday_probs, 10_000):.3f}')
```

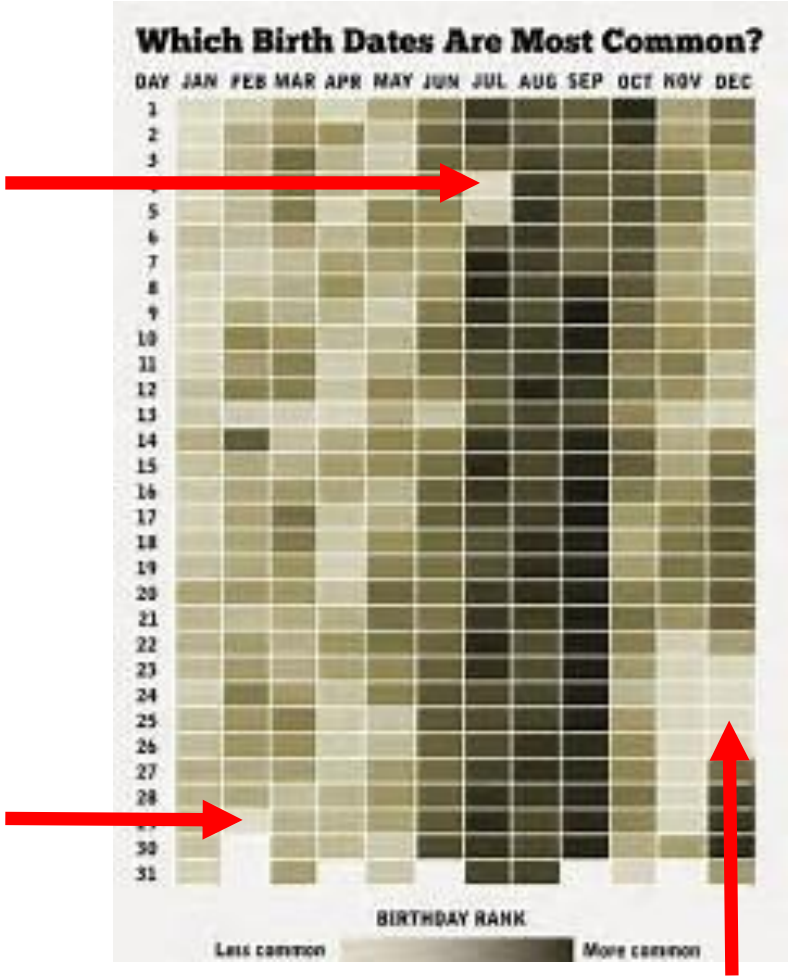
Change 2 to 4

```
Est. prob. of 4 people sharing a birthday, with 2 people: 0.000
Est. prob. of 4 people sharing a birthday, with 4 people: 0.000
Est. prob. of 4 people sharing a birthday, with 8 people: 0.000
Est. prob. of 4 people sharing a birthday, with 16 people: 0.000
Est. prob. of 4 people sharing a birthday, with 32 people: 0.001
Est. prob. of 4 people sharing a birthday, with 64 people: 0.011
Est. prob. of 4 people sharing a birthday, with 128 people: 0.154
Est. prob. of 4 people sharing a birthday, with 256 people: 0.888
```

Simulation is answering a question we cannot answer easily analytically

But still not quite the right question. Assumes all birth dates equally likely.

# All Birth Dates Are NOT Equally Likely

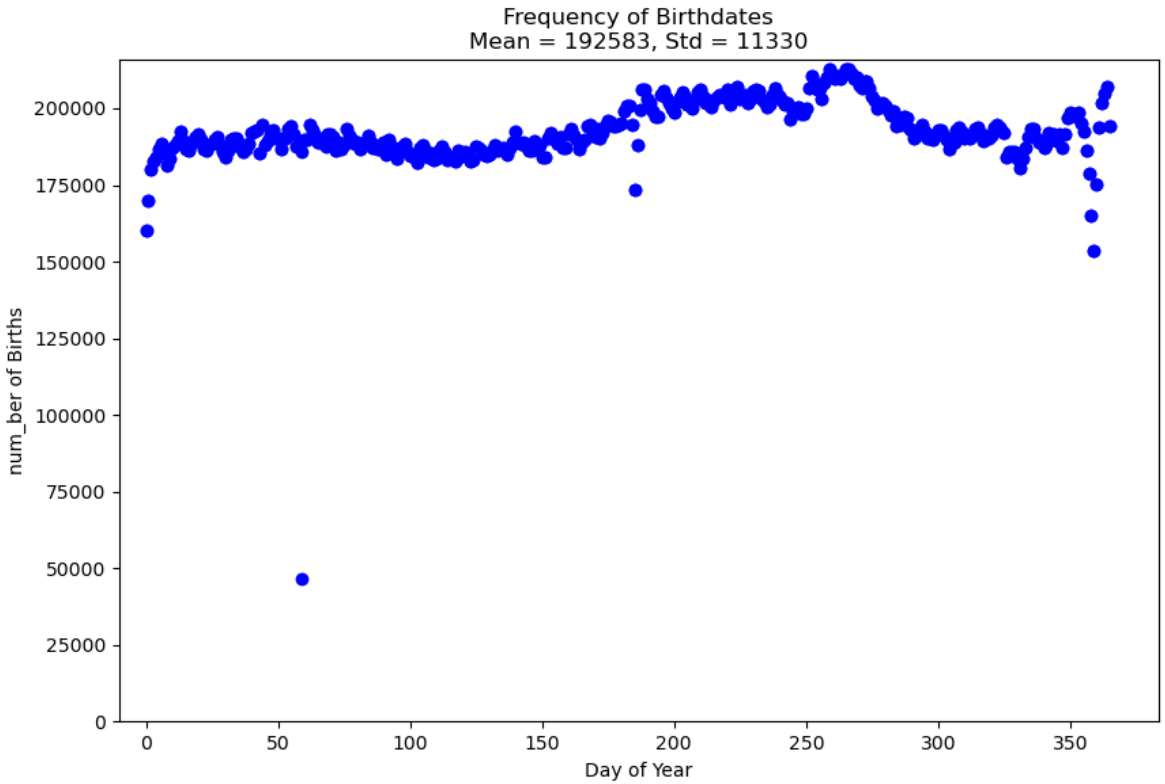



births.csv

```
month,day,births
1,1,160369
1,2,169896
1,3,180036
1,4,182854
1,5,184145
1,6,186726
1,7,188277
1,8,185186
1,9,181511
1,10,183668
...
```

Poll: why are some birthdays less common?

# How Does this Affect Probabilities?




- Do you expect a change?
- Don't even think about an analytic solution
- Adjusting simulation model easy

# Getting the Distribution of Births

```
def get_bdays():
    infile = open('births.csv')
    infile.readline() # discard first line
    num_births = [int(line.split(',')[2][:-1]) for line in infile]
    birthday_probs = np.array(num_births)/num_births.sum()
    return birthday_probs
```

What is the value of sum(birthday_probs)?

# The Simulation (100,000 Trials)

```
np.random.seed(1)
trials = 100_000
birthday_probs = get_bdays()
uniform_probs = np.array([1 for _ in range(366)])/366
num_same = 4
for i in range(4, 9):
    num_people = 2**i
    print(f"For population {num_people}, probability of {num_same} "
          f"shared birthdays:")
    print(f"  Est. for actual distribution: "
          f"{birthday_prob(num_people, num_same, birthday_probs, trials):.5f}")
    print(f"  Est. for uniform distribution: "
          f"{birthday_prob(num_people, num_same, uniform_probs, trials):.5f}")
```

# Results

For population 16, probability of 4 shared birthdays:
  Est. for actual distribution: 0.00003
  Est. for uniform distribution: 0.00004
For population 32, probability of 4 shared birthdays:
  Est. for actual distribution: 0.00087
  Est. for uniform distribution: 0.00057
For population 64, probability of 4 shared birthdays:
  Est. for actual distribution: 0.01192
  Est. for uniform distribution: 0.01152
For population 128, probability of 4 shared birthdays:
  Est. for actual distribution: 0.15601
  Est. for uniform distribution: 0.15276
For population 256, probability of 4 shared birthdays:
  Est. for actual distribution: 0.89009
  Est. for uniform distribution: 0.88680

Note that probability is slightly higher for actual distribution

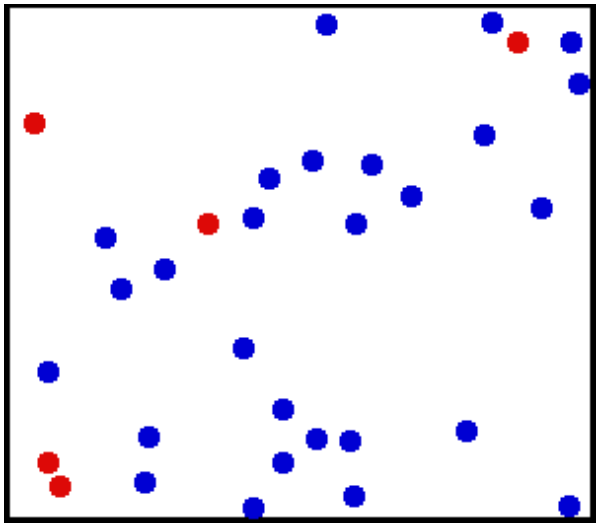
## Simulations Are Used a Lot

- To model systems that are mathematically intractable
- To extract useful intermediate results
- To support iterative development by successive refinement
  - Start with basic system, then incrementally add features
- To support exploring of variations by asking/answering “what if” questions
- Let’s illustrate with random walks

# What are Random Walks?

Method to model a system where:

- Objects start at a location and **choose a random direction** in which to take each step
- Distribution of choice of direction (and speed) can be different
  - For different object types
  - For different environmental conditions

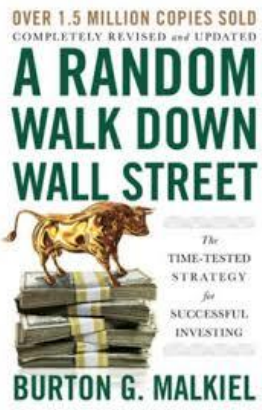


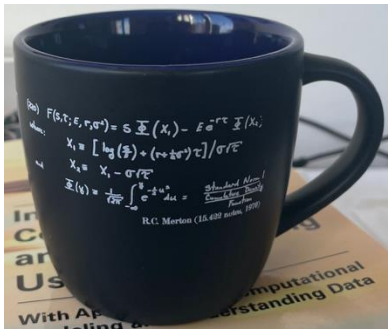
# Random Walks in the World

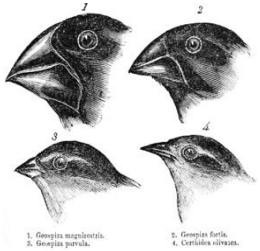
- In modeling stocks, change in price often modeled as a Gaussian random walk
  - Step size chosen from a normal distribution
  - Basic assumption of Black-Scholes-Merton option pricing model

Scholes and Merton won a Nobel Prize for this work

- In population genetics, random walk describes statistical properties of genetic drift
  - Change in frequency of specific gene variant in a population





Darwin's finches

# Why Do We Cover Random Walks?

- Important for modeling behavior in many domains
  - Understanding the stock market, modeling diffusion processes, modeling cell movement, suggesting who to follow on Instagram, etc.
- Solve computational problems (AI, lighting simulation)
- Good illustration of how to use simulations to understand things

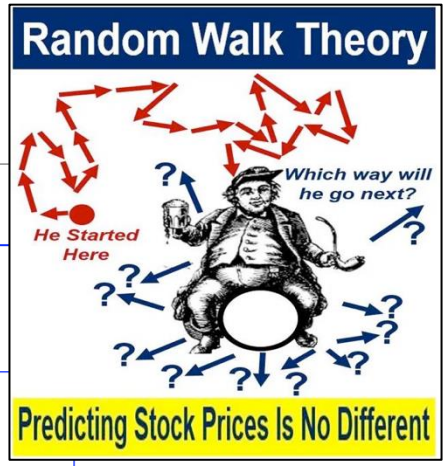
# Our Random Walk

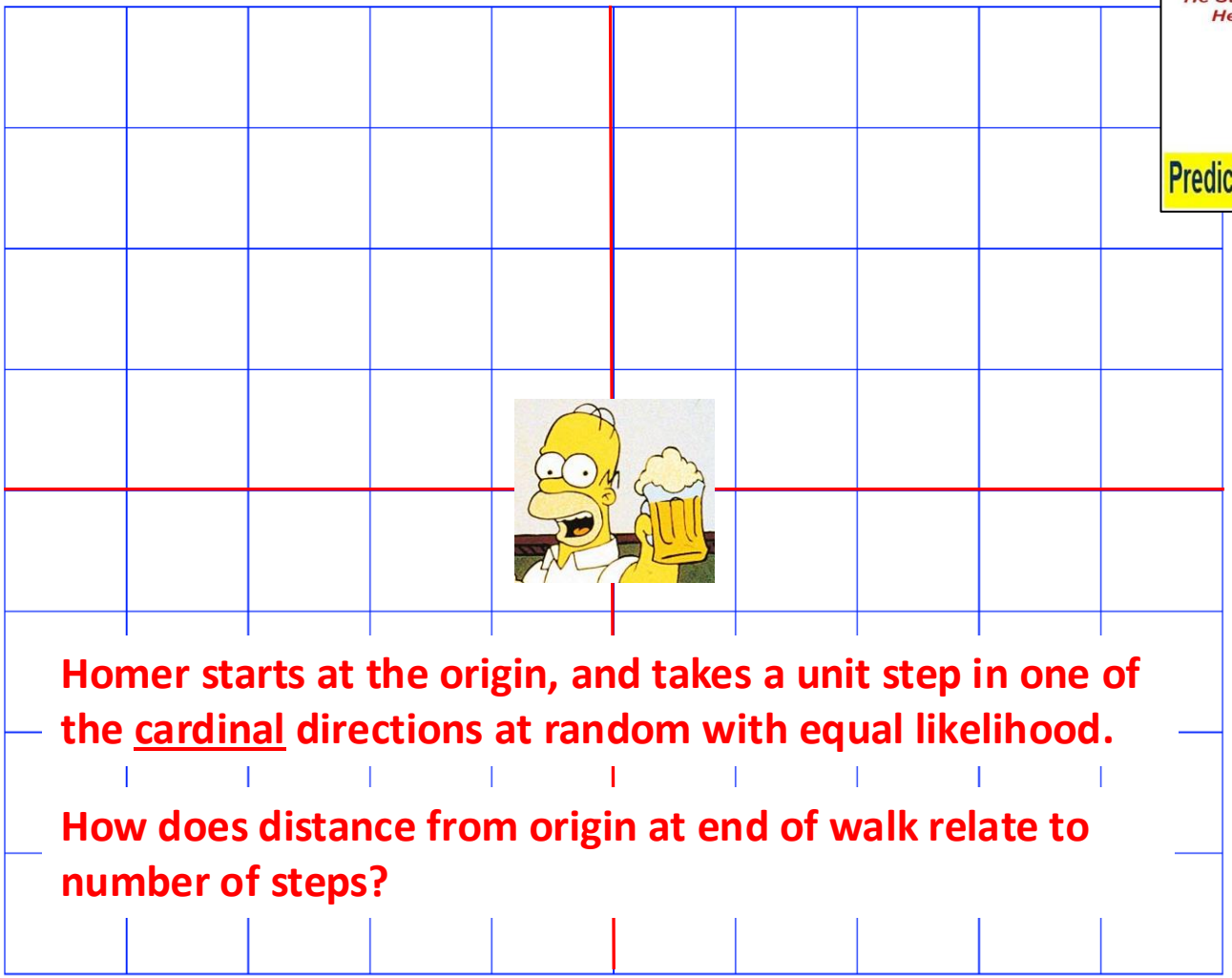
- Often called the drunkard's walk
- A random walk on a two-dimensional surface
  - Walker starts at some initial location
- Each step a fixed distance (initially)
- Step is taken in a direction chosen randomly from a set of choices

Homer's Odysseus – classic wanderer

Homer – Just Odd

# Simplified Drunkward's Walk



**Homer starts at the origin, and takes a unit step in one of the <u>cardinal</u> directions at random with equal likelihood.**

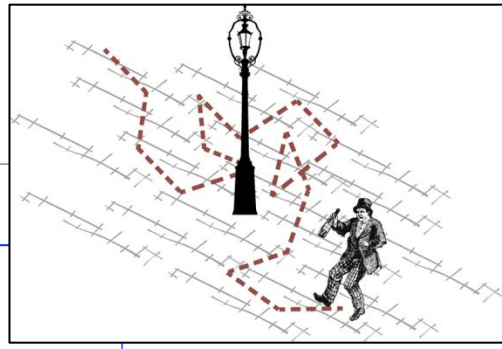
**How does distance from origin at end of walk relate to number of steps?**

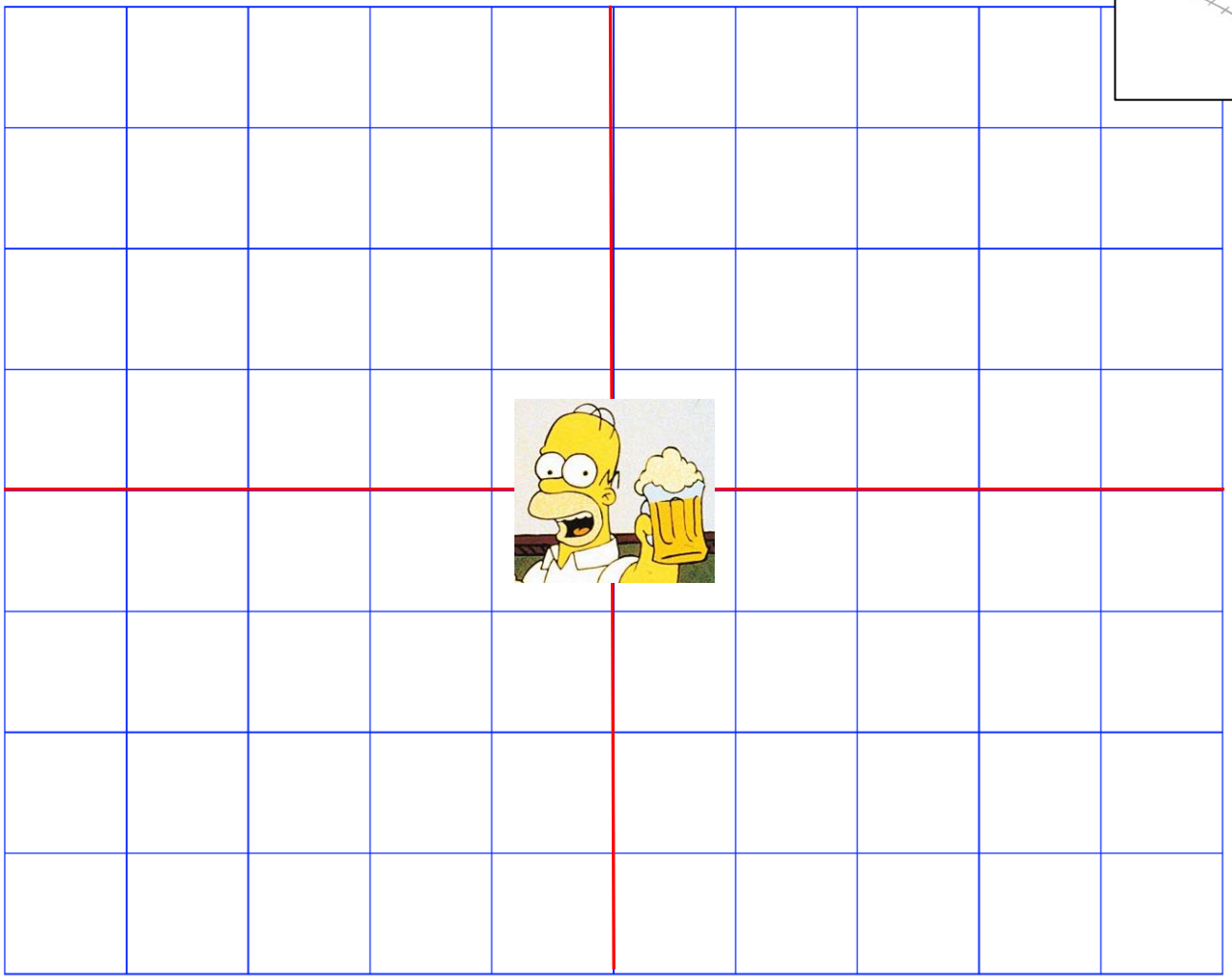
# How Far will Homer Get?

How will the expected distance from the starting point change as the number of steps, d, grows?

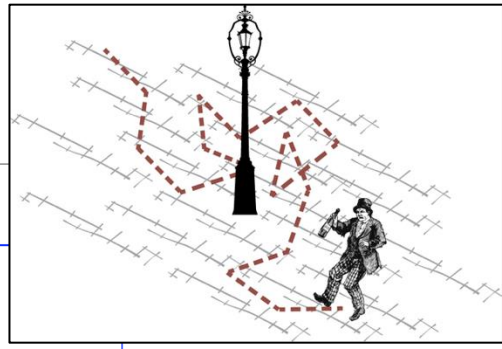
- It won't
- O(log d)
- O(d**0.5)
- O(d)
- O(d**2)
- O(2**d)

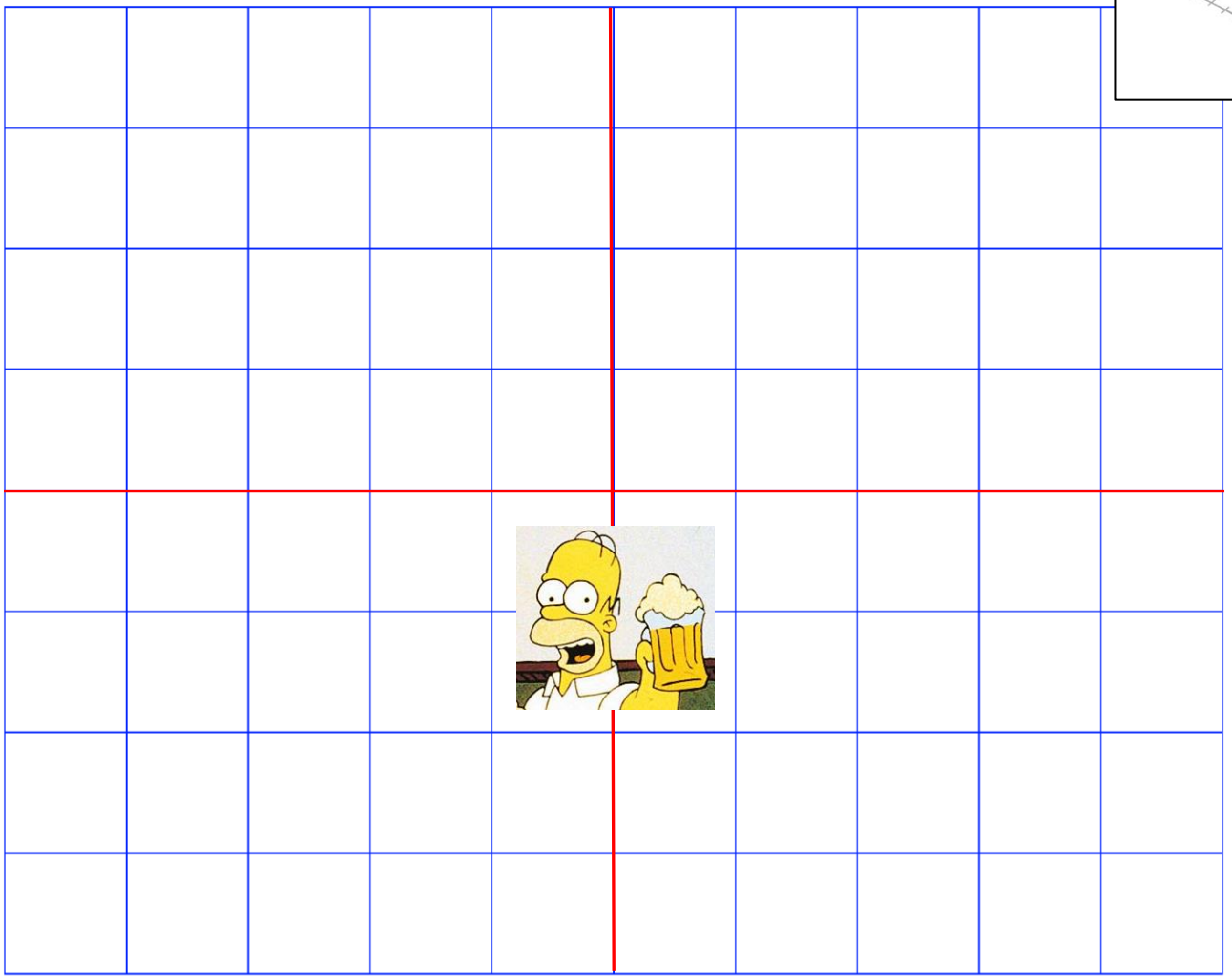
# Hand Simulation



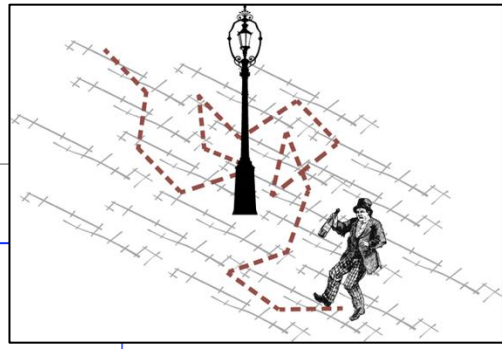


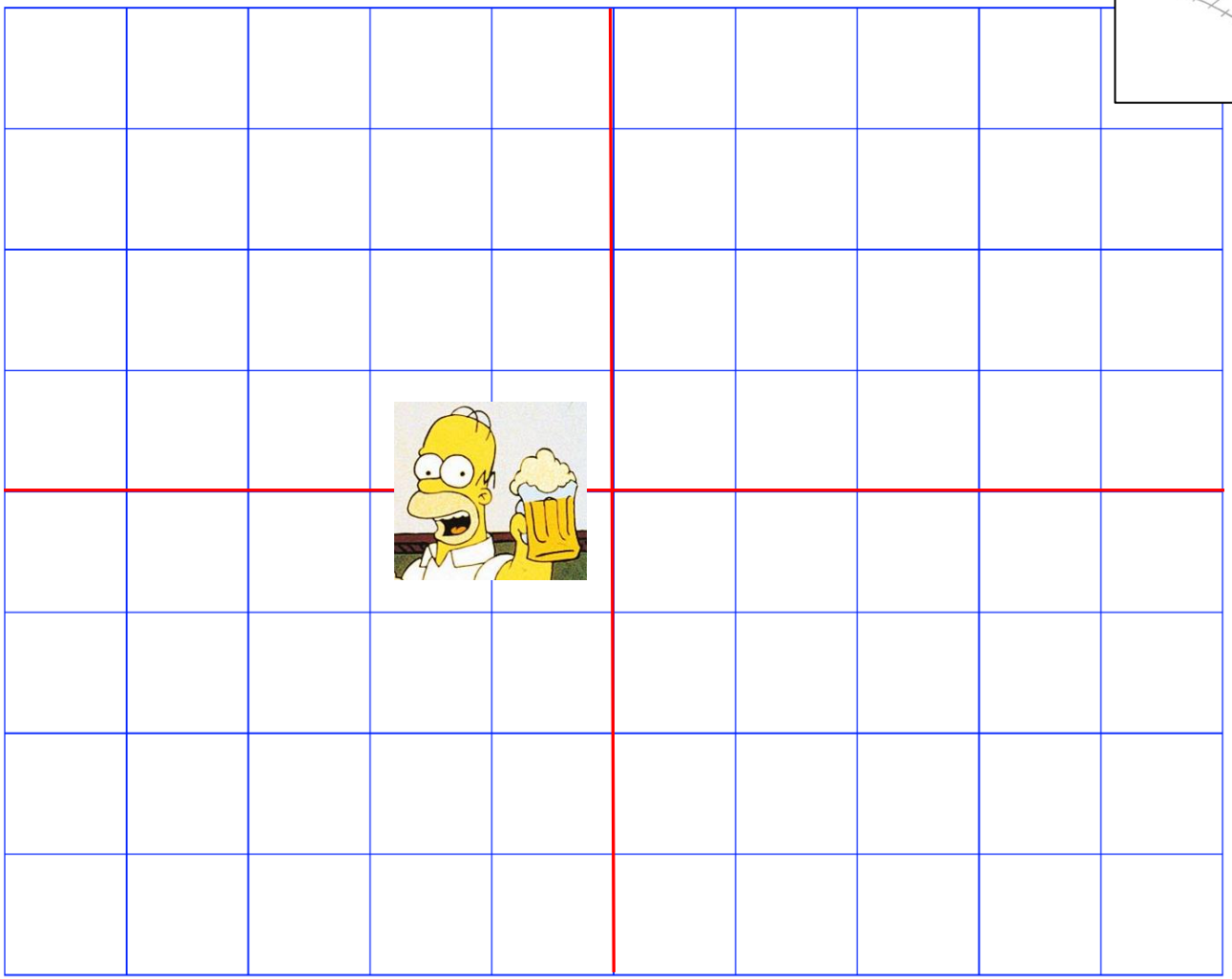
# One Possible First Step



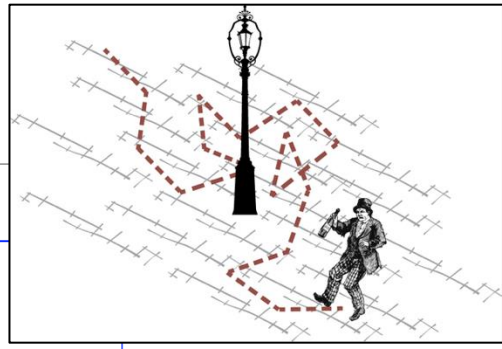


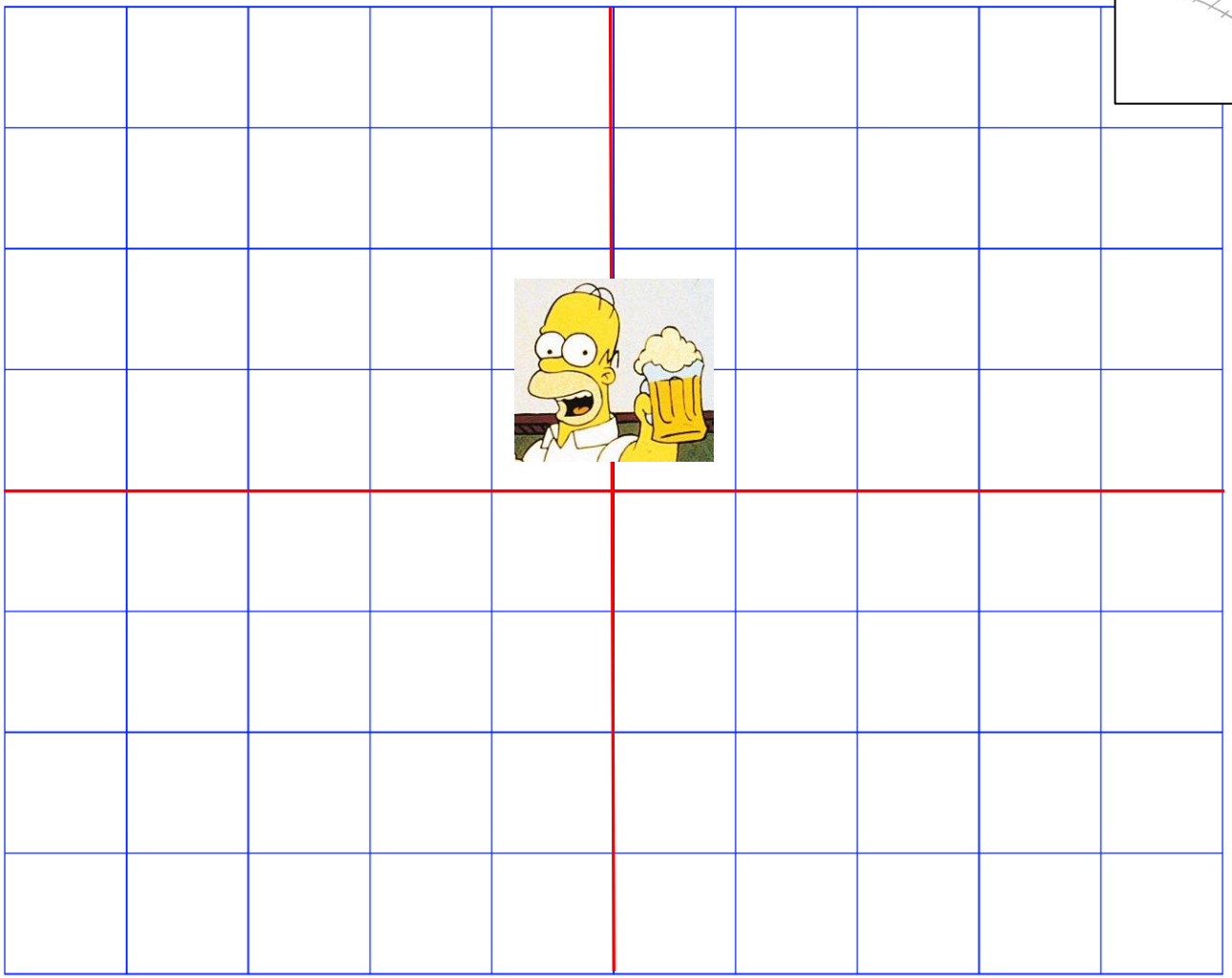
# Another Possible First Step



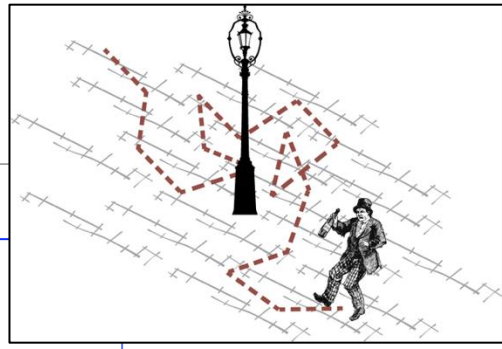


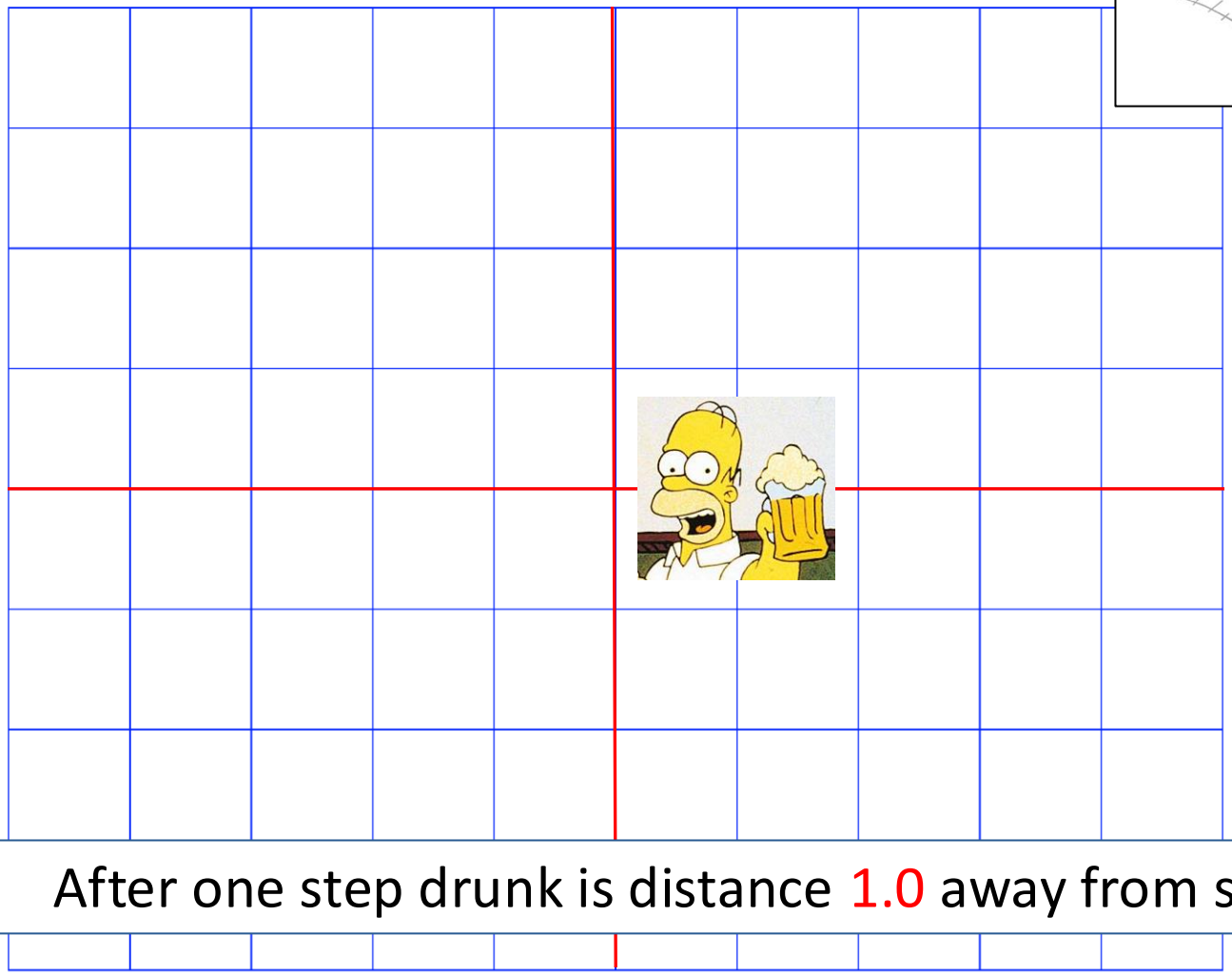
# Yet Another Possible First Step





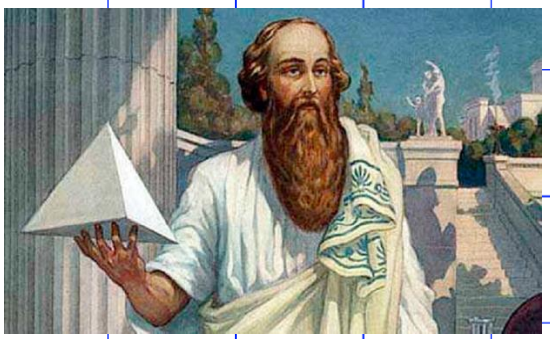
# Last Possible First Step

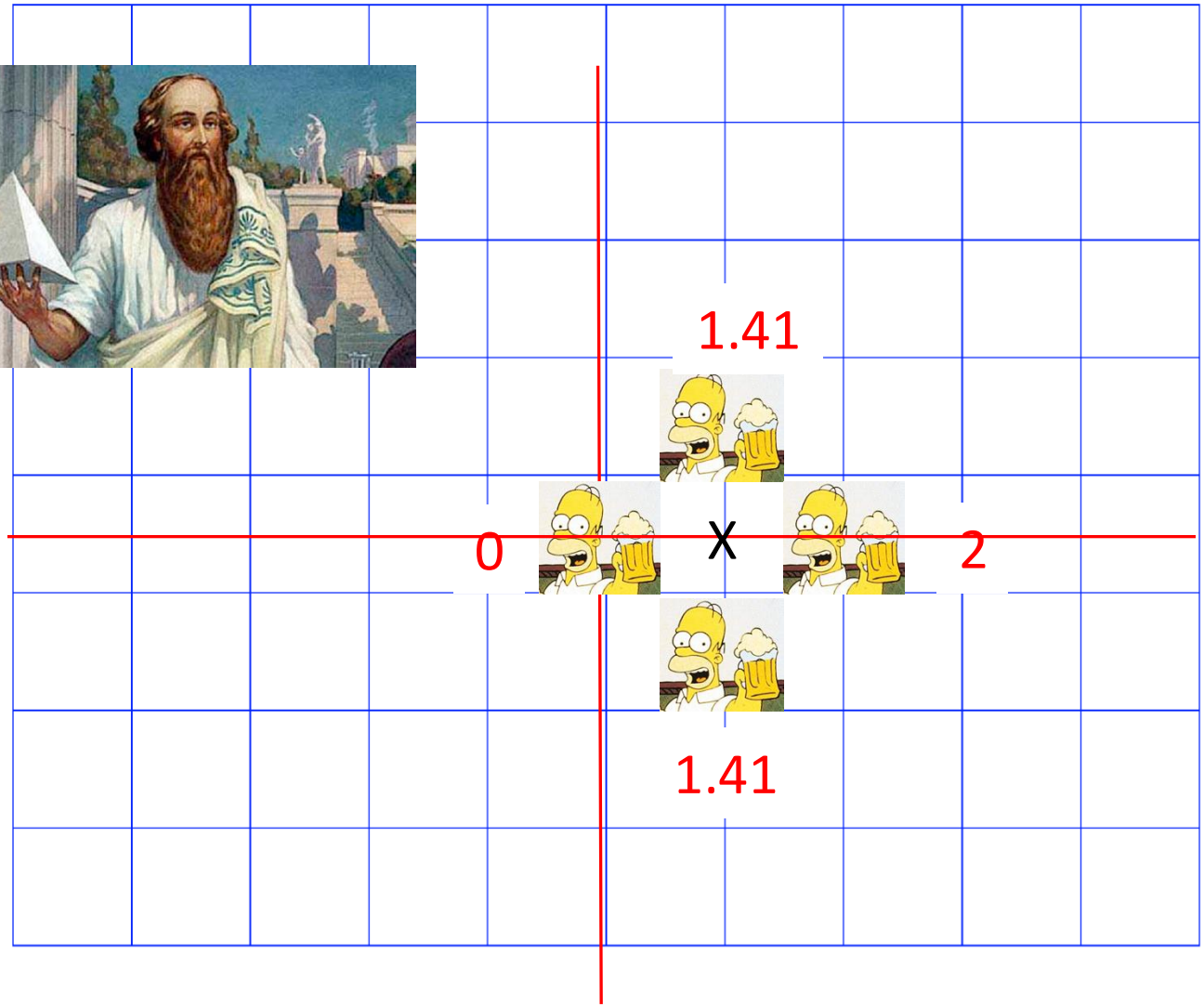




After one step drunk is distance 1.0 away from start

# Possible Locations after Two Steps



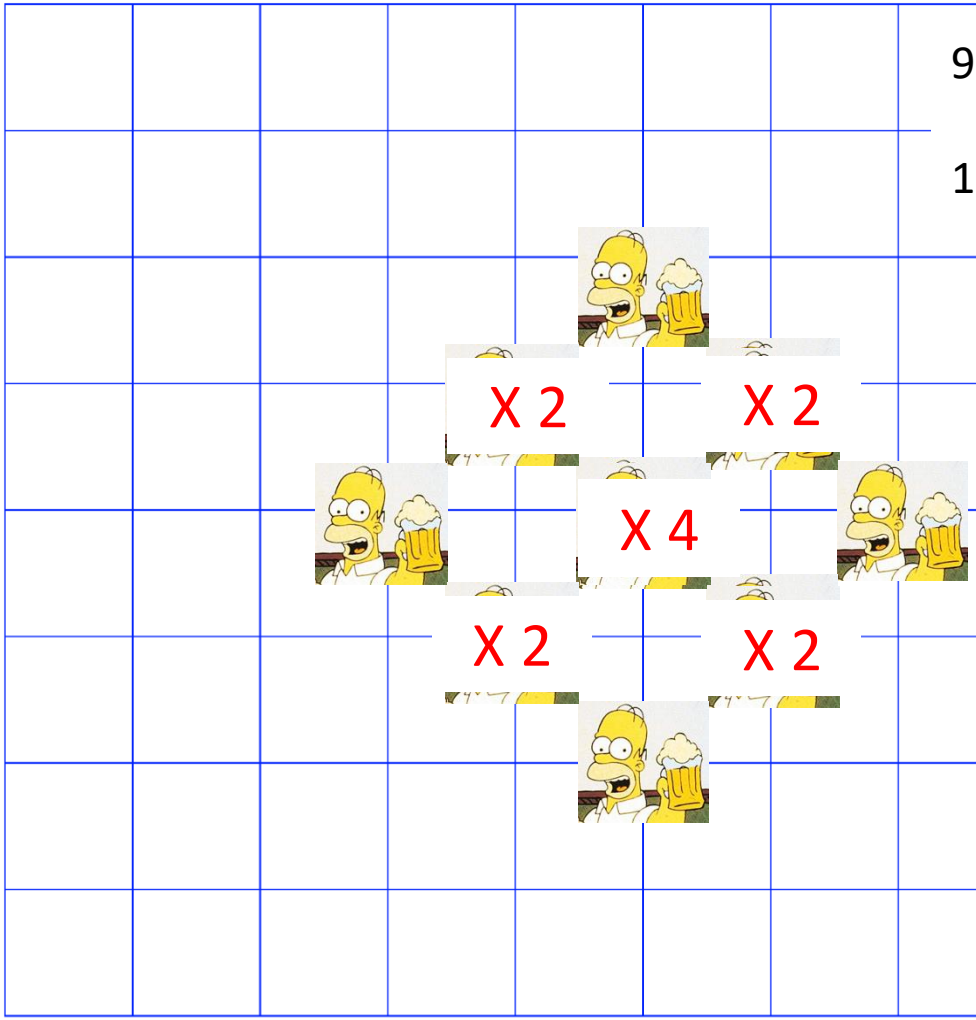



$a^2 + b^2 = c^2$

$c = (a^2 + b^2)^{0.5}$

Assuming 1st step to right

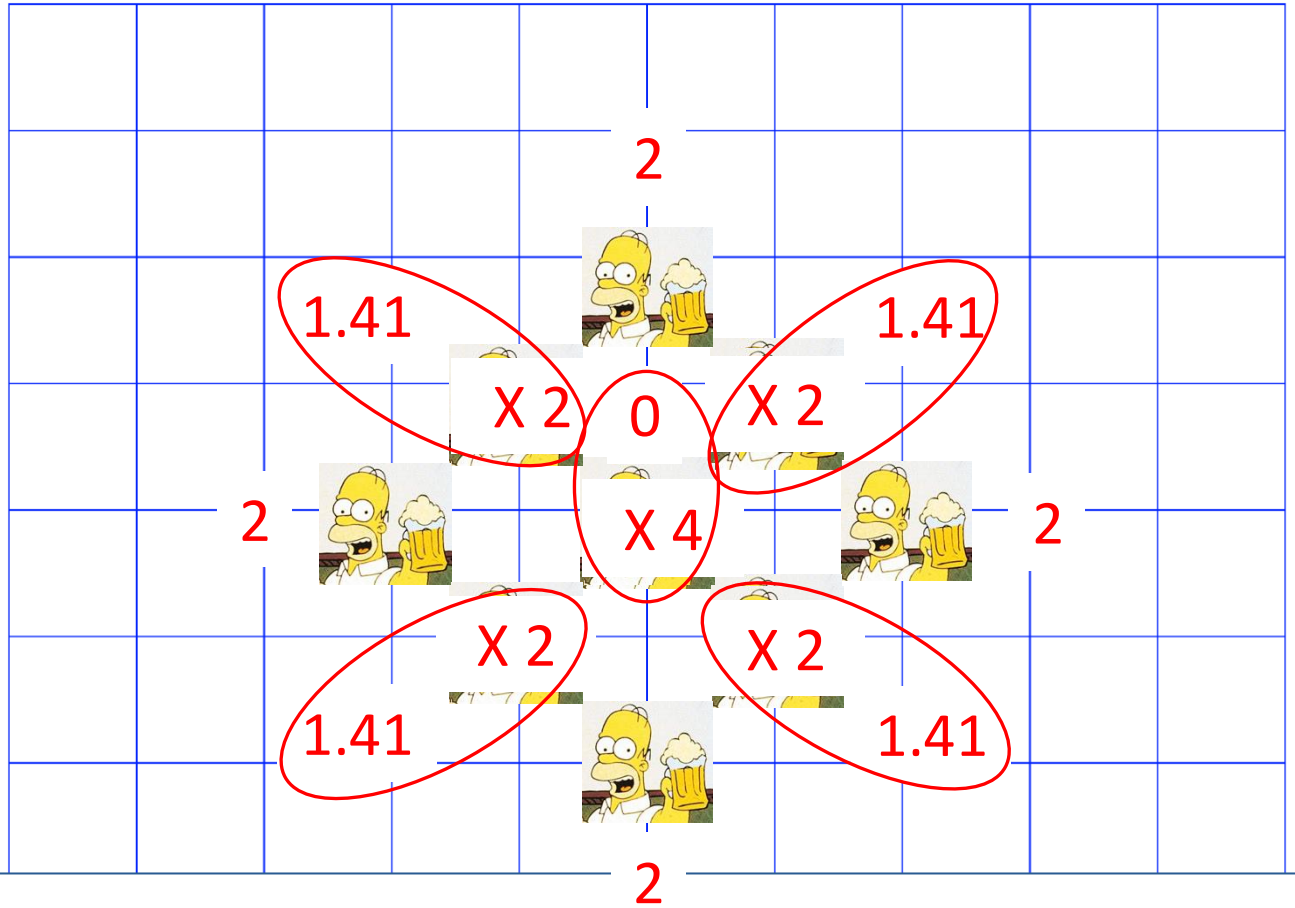
# Possible Distances After Two Steps




9 possible final locations

16 possible different paths

# Possible Distances After Two Steps




Average:

(2 + 2* 1.41 + 2 + 2* 1.41 + 2 + 2* 1.41 + 2 + 2 *1.4 + 4*0)/16

= 1.205

# Expected Distance After 100,000 Steps?

- Can see that expected distance seems to increase with number of steps, and certainly maximum distance does
  - Went from **0** to **1** to **1.205** as expected distance
- But can we accurately model the actual expected distance (i.e., with a mathematical formula)?
  - Seems pretty hard
- What about computing all possible paths, and measuring actual average distance?

# Feasible?

How many possible paths are there after 100,000 steps?

- 100,000
- 400,000
- 100,000**2
- 4**100,000
- 2**100,000

Note, the number of possible paths is different from the number of possible final locations – latter is roughly a mere 100,000**2 = 10B

# Expected Distance after 100,00 steps?

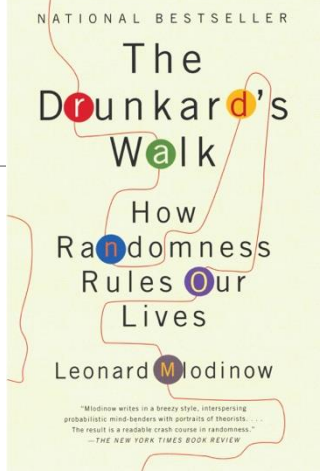
My graph paper isn’t big enough!

# Enumerating all Possibilities Is Not Practical

- Need a different approach to problem
- A probabilistic simulation
  - Won’t yield precisely correct answer
  - But should yield a good approximation

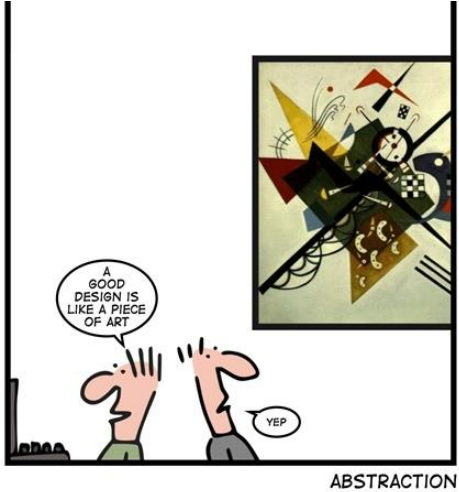
# Five Minute Break

# Structure of Simulation




- Simulate one walk of *d* steps
  - Record distance from start at end of walk
  - Record final location at end of walk
- Simulate *n* such walks, each of *d* steps
  - Record all distances and final locations
- Report average distance from origin over set of *n* walks
  - How does this change as we increase *d*?
- Will come back to final locations over set of walks later

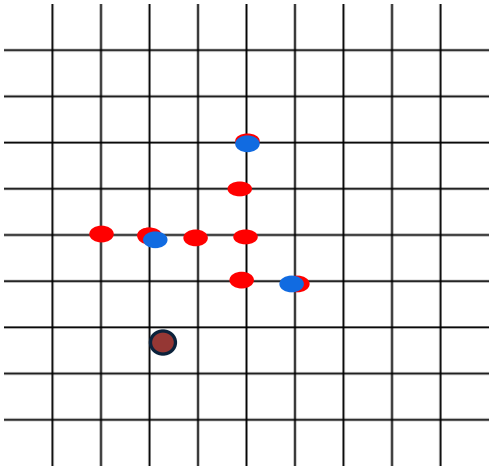
# Implementation




- First, some useful abstractions
- Location—a place
  - An object, like a drunk, has a location
  - Location can change as object moves
- Field—a collection of drunks with locations
  - Store those locations occupied by drunks

# Why Not the Following?

- Field as a two-dimensional array of points, where we store a drunk at a grid point and let drunk move to other grid points
- Wastes a lot of memory because most locations unoccupied
- Problematic for non-cardinal steps since number of possible locations infinite
- Better to build model around locations occupied by drunks, not all possible locations



# Overall plan

- Field
  - Implicit collection of locations
  - Explicitly represent only locations occupied by drunks
  - Associate location directly with a drunk
  - Represent by collection of drunks, not collection of possible locations
- Drunk
  - Can move, change location
- Simulation
  - Create one or more drunks, with initial location
  - Randomly move d steps, record final location, distance from start
  - Do this n times, measure **average** final distance, collect set of final locations

# Class Location, part 1

'Yes, business is brisk – Location, Location, Location!'

```
class Location(object):
    def __init__(self, x, y):
        """x and y are numbers"""
        self.x = x
        self.y = y

    def move(self, deltaX, deltaY):
        """deltaX and deltaY are numbers"""
        return Location(self.x + deltaX,
                        self.y + deltaY)

    def getX(self):
        return self.x

    def getY(self):
        return self.y
```

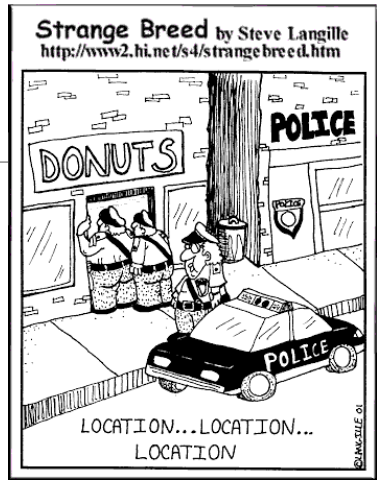
Note that `move` returns a **new** location

Will let x, y, deltaX and deltaY be floats

More general than example

Associate move method with location; since objects have locations, this method says to where an object will be when moved

# Class Location, continued




```
def dist_from(self, other):
    xDist = self.x - other.getX()
    yDist = self.y - other.getY()
    return (xDist**2 + yDist**2)**0.5

def __str__(self):
    return '<' + str(self.x) + ', ' + str(self.y) + '>'
```

# Class Drunk

```
class Drunk(object):
    def __init__(self, name = None):
        """Assumes name is a str"""
        self.name = name

    def __str__(self):
        if self != None:
            return self.name
        return 'Anonymous'
```

Not intended to be useful on its own

A base class from which to inherit

# The Usual Drunk

```
class Usual_drunk(Drunk):
    def take_step(self):
        step_choices = [(0,1), (0,-1),
                        (1,0), (-1,0)]
        return random.choice(step_choices)
```

Inherits from base class of Drunk

Basic activity of a drunk is to wander; take_step will return an x and y tuple, denoting **change** in location

This type of drunk takes a unit step in a cardinal direction, where direction chosen randomly with equal probability

The drunk in our hand simulation

# Another Subclass of Drunk

Inherits from base class of Drunk

```
class Mashochist_drunk(Drunk):
    def take_step(self):
        step_choices = [(0.0,1.1), (0.0,-0.9),
                        (1.0,0.0), (-1.0,0.0)]
        return random.choice(step_choices)
```

Takes a random step in cardinal directions, but if northward, moves a bit further than if southward

# More Subclasses

```
class Liberal_drunk(Drunk):
    def take_step(self):
        step_choices = [(0.0,1.0), (0.0,-1.0),
                        (0.9,0.0), (-1.1,0.0)]
        return random.choice(step_choices)

class Conservative_drunk(Drunk):
    def take_step(self):
        step_choices = [(0.0,1.0), (0.0,-1.0),
                        (1.1,0.0), (-0.9,0.0)]
        return random.choice(step_choices)
```

# Class Field, part 1

Field: a collection of drunks, each of which has a location

```
class Field(object):
    def __init__(self):
        self.drunks = {}

    def add_drunk(self, drunk, loc):
        if drunk in self.drunks:
            raise ValueError('Duplicate drunk')
        else:
            self.drunks[drunk] = loc

    def get_loc(self, drunk):
        if drunk not in self.drunks:
            raise ValueError('Drunk not in field')
        return self.drunks[drunk]
```

Dictionary mapping Drunk to Location; Allows multiple drunks in a field

Use dictionary lookup to find location of drunk

Note: using drunk **object,** not drunk name as key in dictionary. Why is this okay?

# Class Field, continued

```
def move_drunk(self, drunk):
    if drunk not in self.drunks:
        raise ValueError('Drunk not in field')
    x_dist, y_dist = drunk.take_step()
    # use move method of Location to set new location
    self.drunks[drunk] =\
        self.drunks[drunk].move(x_dist, y_dist)
```

Returns a tuple

Store that new location in dictionary as entry associated with drunk (as key)

Thus, Field is maintained as collection of drunks, whose associated value is their location

Dealing with Location object

Get cordinates of new location

Remember that move returns a new location by adding deltas to current location values

# Simulating a Single Walk

```
def walk(f, d, num_steps):
    """Assumes: f a Field, d a Drunk in f,
                and num_steps an int >= 0.
       Moves d num_steps times, and returns the distance between the
       final location and the location at the start of the walk."""
    start = f.get_loc(d)
    for s in range(num_steps):
        f.move_drunk(d)
    return start.dist_from(f.get_loc(d))
```

Get initial location of drunk

Iteratively move drunk `d` in field `f`

Get final location of drunk d

Use method on location of drunk `d` to get distance from `start` location

# Simulating Multiple Walks

```
def sim_walks(num_steps, num_trials, d_class):
    """Assumes num_steps an int >= 0, num_trials an int > 0,
       d_class a subclass of Drunk
      Simulates num_trials walks of num_steps steps each.
      Returns a list of the final distances for each trial"""
    Homer = d_class('Homer')
    origin = Location(0, 0)
    distances = []
    for t in range(num_trials):
        f = Field()
        f.add_drunk(Homer, origin)
        distances.append(round(walk(f, Homer, num_trials), 1))
    return distances
```

The name of a **class**, not an instance, leads to a generalized function

Can use same code to test different kinds of drunks

Insert drunk into field, at origin

Simulate walk, and save final distance from starting point

# Putting It All Together

```
def drunk_test(walk_lengths, num_trials, d_class):
    """Assumes walk_lengths a sequence of ints >= 0
         num_trials an int > 0, d_class a subclass of Drunk
       For each number of steps in walk_lengths, runs
       sim_walks with num_trials walks and prints results"""
    for num_steps in walk_lengths:
        distances = sim_walks(num_steps, num_trials, d_class)
        print(f'{d_class.__name__} random walk of {num_steps:,} steps')
        print(f' Mean = {sum(distances)/len(distances):.2f}')
        print(f' Max = {max(distances):.2f}, Min = {min(distances):.2f}')
```

# Let's Try It

How many steps to use

```
drunk_test((10, 100, 1000, 10000), 100,
           Usual_drunk)
```

Number of trials

```
Usual_drunk random walk of 10 steps
 Mean = 8.634
 Max = 21.6 Min = 1.4
Usual_drunk random walk of 100 steps
 Mean = 8.57
 Max = 22.0 Min = 0.0
Usual_drunk random walk of 1000 steps
 Mean = 9.206
 Max = 21.6 Min = 1.4
Usual_drunk random walk of 10000 steps
 Mean = 8.727
 Max = 23.5 Min = 1.4
```

Should mean distance be same for different numbers of steps?

Should mean distance be almost 9 after 10 steps?

Does this pass the smell test?

# Let's Try a Smoke Test (aka Sanity Check)

- Try on cases where we think we know the answer
  - A very important precaution!

# Sanity Check

```
drunk_test((0, 1, 2), 100, UsualDrunk)
```

Try simulations of 0, 1 and 2 steps

Know from our hand simulation that distances should be roughly 0, 1, and 1.205

# Sanity Check

```
Usual_drunk random walk of 0 steps
 Mean = 8.634
 Max = 21.6 Min = 1.4
Usual_drunk random walk of 1 steps
 Mean = 8.57
 Max = 22.0 Min = 0.0
Usual_drunk random walk of 2 steps
 Mean = 9.206
 Max = 21.6 Min = 1.4
```

Try simulations of 0, 1 and 2 steps

After 0 steps, on average we are 8.6 units away?

# Simulating Multiple Walks

```
def sim_walks(num_steps, num_trials, d_class):
    """Assumes num_steps an int >= 0, num_trials an int > 0,
         d_class a subclass of Drunk
       Simulates num_trials walks of num_steps steps each.
       Returns a list of the final distances for each trial"""
    Homer = d_class('Homer')
    origin = Location(0, 0)
    distances = []
    for t in range(num_trials):
        f = Field()
        f.add_drunk(Homer, origin)
        distances.append(round(walk(f, Homer,
                                    num_trials), 1))
    return distances
```

num_steps

# Sanity Check

Usual_drunk random walk of 0 steps
Mean = 0.0
Max = 0.0 Min = 0.0
Usual_drunk random walk of 1 steps
Mean = 1.0
Max = 1.0 Min = 1.0
Usual_drunk random walk of 2 steps
Mean = 1.218
Max = 2.0 Min = 0.0

# Value of Sanity Checks

- Usually build simulations to find answers to questions whose answer we don't known
- When testing code, always good to ask if results are plausible – do they pass the smell test?
- If they don't, then design tests where you know what the result should be
- Once you have debugged your code, rerun the original tests to be sure they now seem reasonable

# Let's Try It

```
drunk_test((10, 100, 1000, 10000), 100,
           Usual_drunk)
```

```
Usual_drunk random walk of 10 steps
 Mean = 2.863
 Max = 7.2 Min = 0.0
Usual_drunk random walk of 100 steps
 Mean = 8.296
 Max = 21.6 Min = 1.4
Usual_drunk random walk of 1000 steps
 Mean = 27.297
 Max = 66.3 Min = 4.2
Usual_drunk random walk of 10000 steps
 Mean = 89.241
 Max = 226.5 Min = 10.0
```

3.16

10.0

31.6

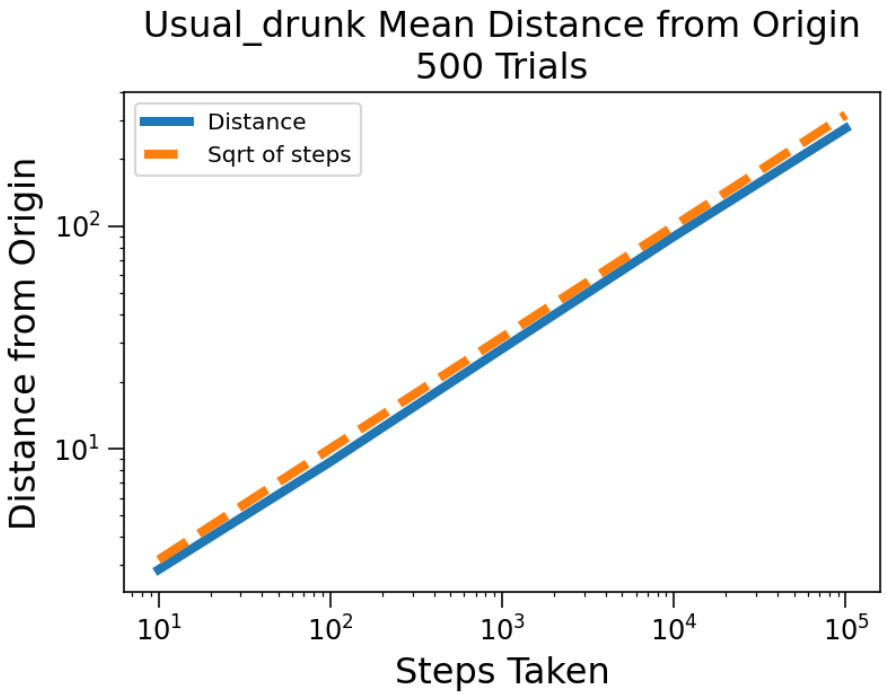
100.0

~square root of number of steps

# Visualizing the Trend

- Simulated walks of different lengths for Usual_drunk
- Seeing distances as numbers helps
- But, better to **plot** distance at end of each length walk for each kind of drunk

Sheds light on why molecular diffusion leads to slow mixing




Note that sqrt is not a perfect fit

More on this in next lecture

# Wrapping Up for Today

- Variance, standard deviation, coefficient of determination, empirical rule
- Using simulation to understand complex situations
  - Variants of birthday problem, random walks
- Tips on construction of simulations
  - Move from simple to more complex
  - Test on obvious cases
- Next lecture
  - A bit more on the drunkards walk
  - Using random walks to simulate behavior of gasses
  - Using random computations to address problems that are not inherently random